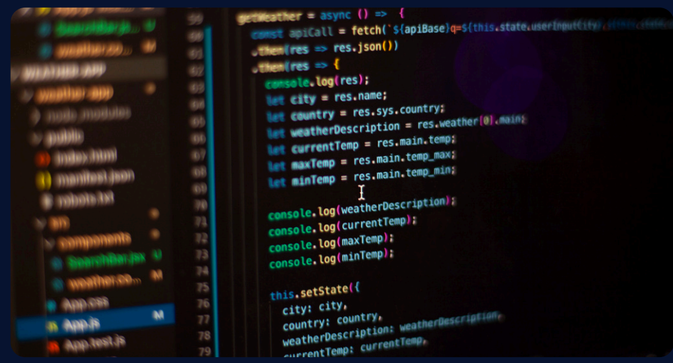
UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL





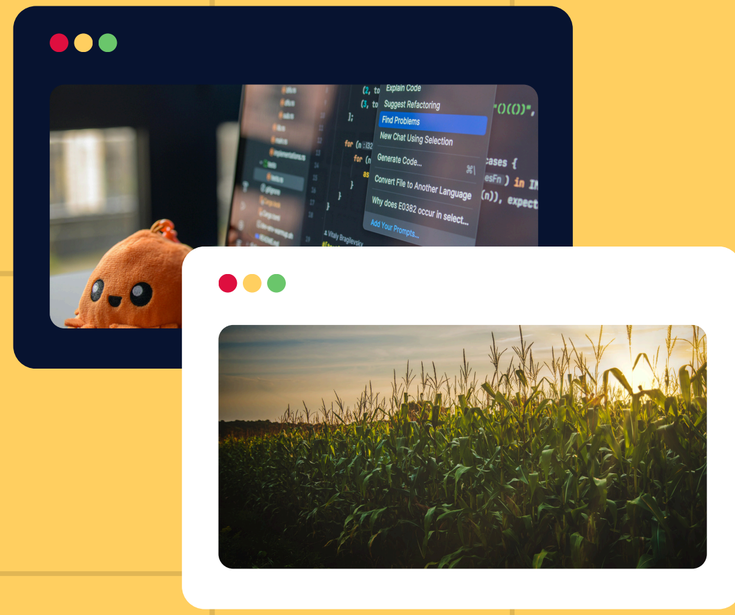
Administración de Campos

Implementación de un Caso de Uso

UTN·FRC
DSI

Cátedra de Diseño de Sistemas de Información - UTN FRC/FRVM - 2026

Contenidos



Caso de Uso - Registrar Laboreo en Lotes
Consideraciones de Implementación
Implementación del caso de uso
Links de Utilidad

UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI

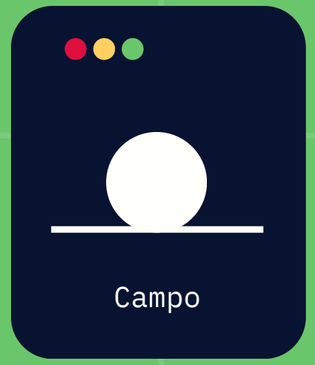
Caso de Uso Registrar Laboreo en Lotes

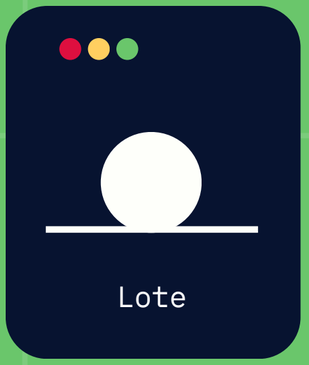


UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI

Caso de Uso - Registrar Laboreo en Lotes

Nombre del Caso de uso: Registrar Laboreos en Lotes	**Nro. de Orden**: 1
Prioridad: ☒ Alta ☐ Media ☐ Baja	
Complejidad: ☐ Simple ☒ Mediano ☐ Complejo ☐ Muy Complejo ☐ Extremadamente Complejo	
Actor Principal: Encargado de Laboreos (EL)	**Actor Secundario**: no aplica
Tipo de Caso de uso: ☒ Concreto ☐ Abstracto	
Objetivo: Registrar uno o más laboreos realizados para cada Lote seleccionado de un campo.	
Flujo Básico	
1. **EL**: selecciona la opción **Registrar Laboreos en un Lote**	
2. **Sistema**: muestra el nombre de todos los campos habilitados y solicita se seleccione uno.	
3. **EL**: selecciona el campo para el cual quiere registrar el laboreo	
4. **Sistema**: busca los lotes que corresponden a ese campo que tengan Proyecto de Cultivo vigente (significa que no esté en estado final) y encuentra al menos uno. De cada uno de ellos muestra el número de lote y la fecha de Inicio del proyecto vigente y solicita que se seleccionen los lotes para los que quiere registrar laboreos.	
5. **EL**: selecciona uno o más lotes.	
6. **Sistema: Para cada lote elegido** muestra el nombre del cultivo y los laboreos efectuados en ese lote, correspondientes al proyecto vigente.	
7. **Sistema**: busca y muestra los tipos de laboreo aplicables al cultivo.	
8. **Sistema**: **Para cada laboreo** que se desea registrar solicita que seleccione alguno.	
9. **EL:** selecciona un laboreo.	
10. **Sistema:** solicita se ingrese la fecha y hora de comienzo, fecha y hora de fin del laboreo.	
11. **EL:** ingresa la fecha y hora de comienzo, fecha y hora de fin del laboreo.	
12. **Sistema:** valida las fechas y horas de inicio y fin del laboreo y son correctos. **(ver observación 2)**	
13. **Sistema**: muestra los empleados y solicita se seleccione el empleado que efectuó el laboreo.	
14. **EL**: selecciona el empleado.	
15. **Sistema**: **Para cada laboreo a registrar** solicita confirmación para efectuar el registro.	
16. **EL:** confirma el registro.	
17. **Sistema**: registra el laboreo realizado con los siguientes datos: fecha comienzo, hora comienzo, fecha fin, hora fin, empleado que lo efectuó, laboreo realizado.	
18. **Sistema:** Valida que el tipo de laboreo no sea siembra ni cosecha.	
19. **EL:** Selecciona la opción para finalizar el registro de laboreos. Fin del caso de uso.	
Flujos Alternativos	
A1: No existe ningún proyecto de cultivo vigente para los lotes seleccionados.	
A2: El actor decide cancelar el caso de uso.	
A3: El actor no confirma registro de laboreos.	
A4: El tipo de laboreo es siembra.	
A5: El tipo de laboreo es cosecha.	
A6: Las fechas y/u horas de inicio y fin de laboreo no son correctas.	
Observaciones:	
1. El actor puede cancelar el caso de uso en cualquier momento eligiendo la opción para tal fin.	
2. Valida fecha de inicio menor que la fecha de fin y que no sean mayores a la fecha del día.	











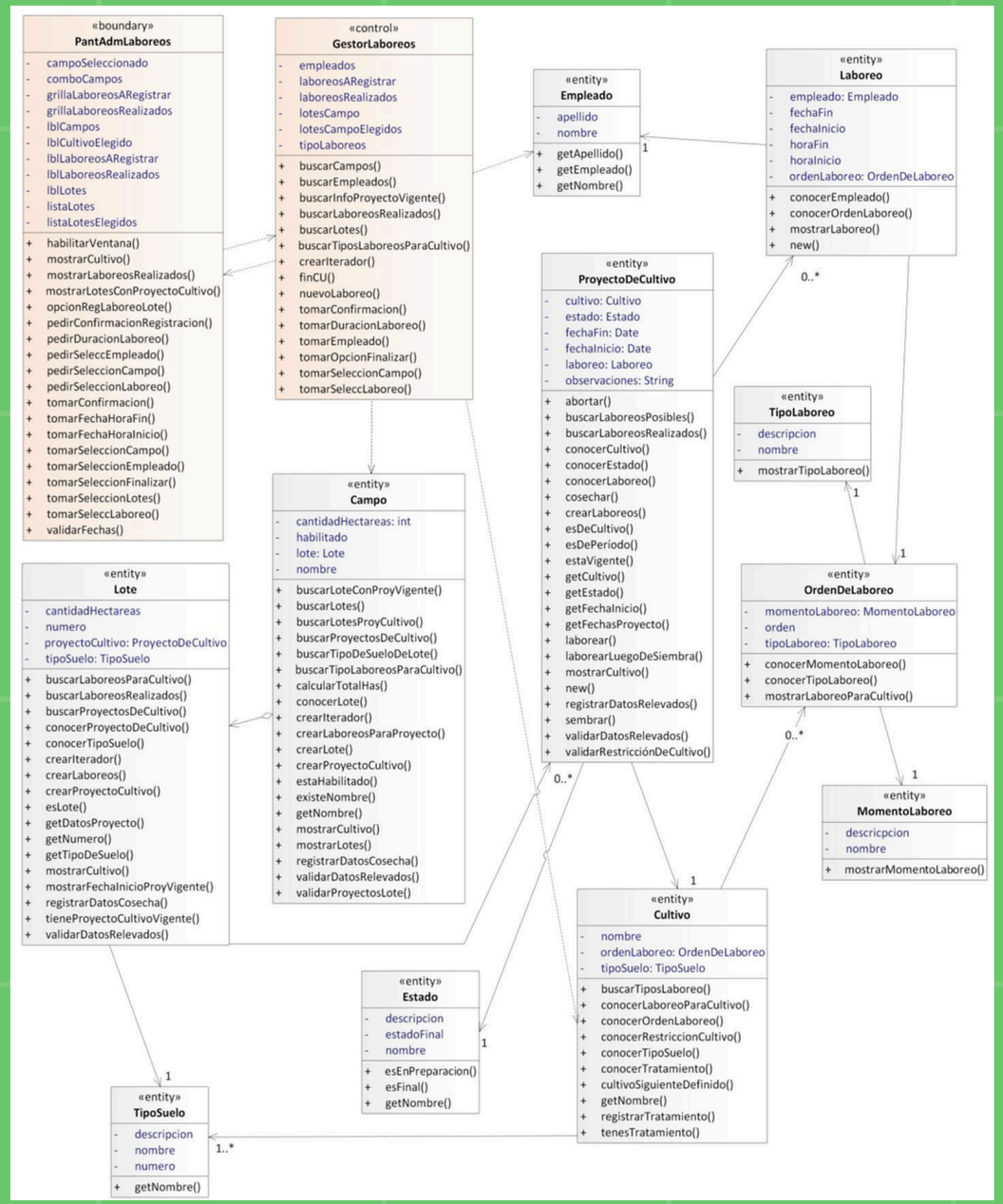


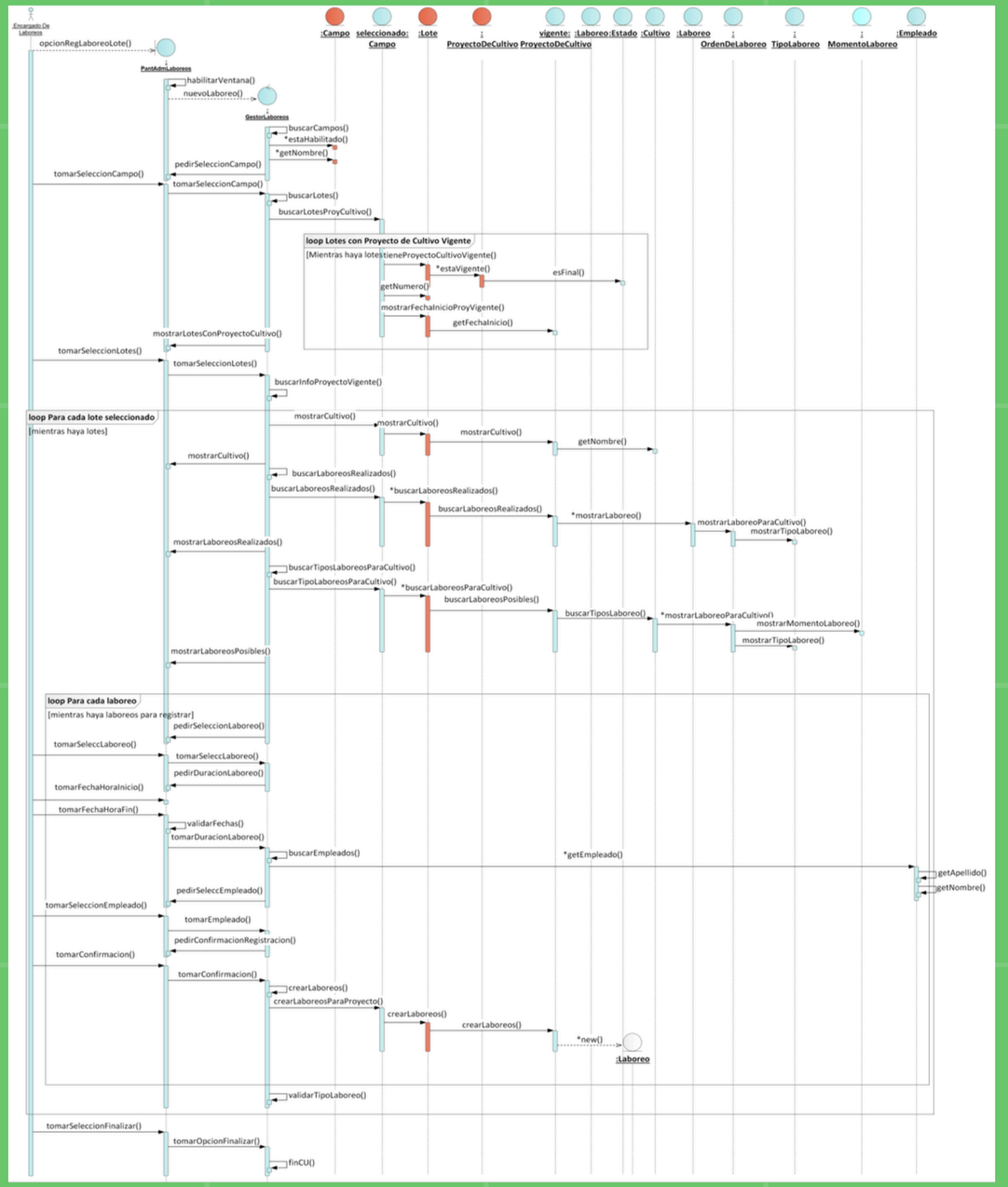






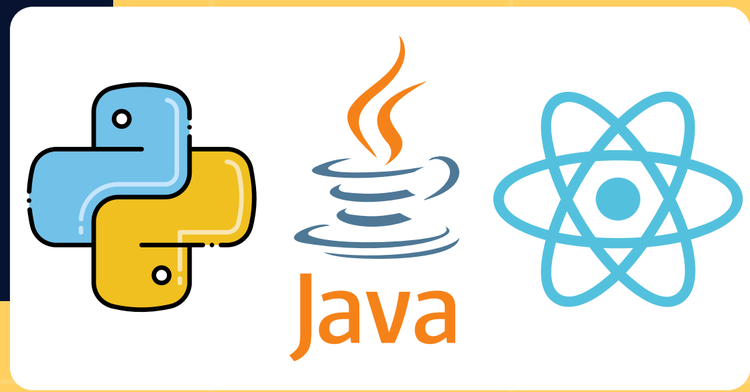
Caso de Uso - Registrar Laboreo en Lotes





UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI

Consideraciones de Implementación



UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI

Consideraciones de Implementación

- El proyecto se encuentra en Java (con Springboot) y Python (con Django) para el backend y React para el frontend.
- A los fines de simplificar lo que se quiere demostrar, se han hecho algunas modificaciones con respecto a la vista dinámica original. Será aclarado en cada apartado.
- Para correr el caso de uso, se debe ejecutar el frontend y uno de los backends (Java/Java con Persistencia/ Python).
- Se encuentran disponibles tres versiones del backend:
  - Java con objetos en memoria
  - Java con persistencia implementada (JPA/H2)
  - Python con persistencia implementada (Django ORM/SQLite)
- El objetivo de esta implementación es demostrar cómo se implementan las clases en un lenguaje orientado a objetos, utilizando un enfoque orientado a una API REST, con lenguajes familiares para el estudiante.
- Para los ejemplos, se tomará de base la implementación hecha en Java no persistente, a los fines de no complejizar la explicación.

Correr Front + Java In Memory

```
# Terminal 1: Backend Java
cd java
mvn spring-boot:run

# Terminal 2: Frontend
cd front
npm install
npm run dev
```

Correr Front + Java con Persistencia

```
# Terminal 1: Backend Java con H2
cd java-persistencia
mvn spring-boot:run

# Terminal 2: Frontend
cd front
npm install
npm run dev
```

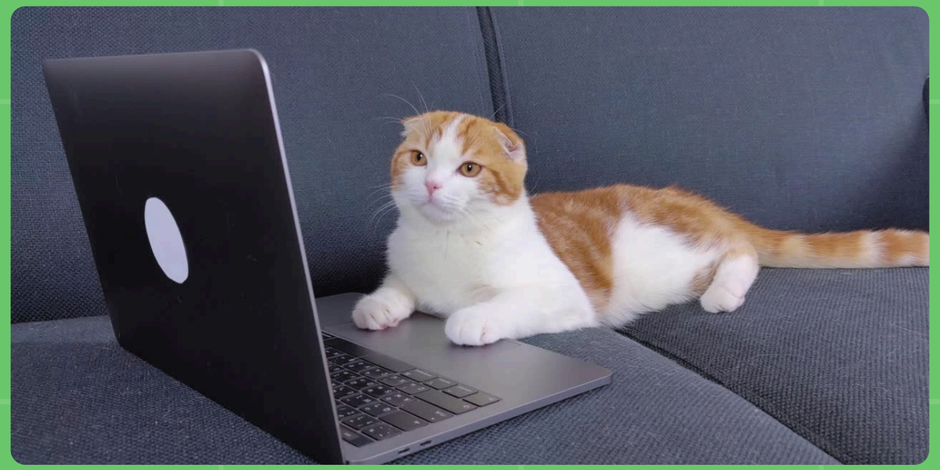
Correr Front + Python

```
# Terminal 1: Backend Python
cd python
pip install -r requirements.txt
python3 iniciar_servidor.py

# Terminal 2: Frontend
cd front
npm install
npm run dev
```



Implementación del Caso de Uso



UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI

Implementación del Caso de Uso

Clase Campo

```java
public class Campo {
    private String nombre;
    private Double cantidadHectareas;
    private Boolean habilitado;
    private List<Lote> lotes;

    public Campo(String nombre, Double cantidadHectareas, Boolean
    habilitado, List<Lote> lotes) {
        this.nombre = nombre;
        this.cantidadHectareas = cantidadHectareas;
        this.habilitado = habilitado;
        this.lotes = lotes;
    }

    public String getNombre() {
        return nombre;
    }

    public Double getCantidadHectareas() {
        return cantidadHectareas;
    }
```

Clase Lote

```java
public class Lote {
    private Integer numero;
    private Double cantidadHectareas;
    private TipoSuelo tipoSuelo;
    private List<ProyectoDeCultivo> proyectosCultivo;

    public Lote(Integer numero, Double cantidadHectareas, TipoSuelo tipoSuelo) {
        this.numero = numero;
        this.cantidadHectareas = cantidadHectareas;
        this.tipoSuelo = tipoSuelo;
        this.proyectosCultivo = new ArrayList<>();
    }

    public Integer getNumero() {
        return numero;
    }

    public Double getCantidadHectareas() {
        return cantidadHectareas;
    }
```

Clase ProyectoDeCultivo

```java
public class ProyectoDeCultivo {
    private Cultivo cultivo;
    private Estado estado;
    private LocalDate fechaInicio;
    private LocalDate fechaFin;
    private String observaciones;
    private List<Laboreo> laboreos;

    public ProyectoDeCultivo(Cultivo cultivo, Estado estado, LocalDate
    fechaInicio, LocalDate fechaFin, String observaciones) {
        this.cultivo = cultivo;
        this.estado = estado;
        this.fechaInicio = fechaInicio;
        this.fechaFin = fechaFin;
        this.observaciones = observaciones;
        this.laboreos = new ArrayList<>();
    }
```

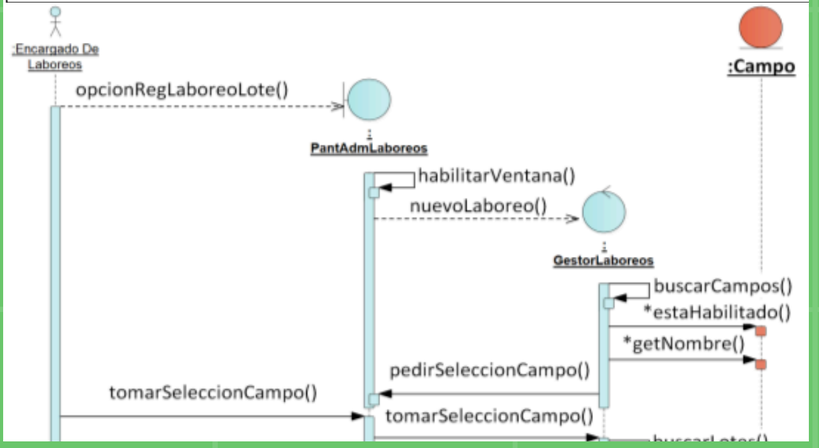
Clase Laboreo

```java
public class Laboreo {
    private LocalDate fechaInicio;
    private LocalDate fechaFin;
    private LocalTime horaInicio;
    private LocalTime horaFin;
    private Empleado empleado;
    private OrdenDeLaboreo ordenLaboreo;

    public Laboreo(LocalDate fechaInicio, LocalDate fechaFin, LocalTime
    horaInicio, LocalTime horaFin, Empleado empleado, OrdenDeLaboreo
    ordenLaboreo) {
        this.fechaInicio = fechaInicio;
        this.fechaFin = fechaFin;
        this.horaInicio = horaInicio;
        this.horaFin = horaFin;
        this.empleado = empleado;
        this.ordenLaboreo = ordenLaboreo;
    }
```

Implementación del Caso de Uso

Flujo Básico
1. **EL**: selecciona la opción **Registrar Laboreos en un Lote**
2. **Sistema**: muestra el nombre de todos los campos habilitados y solicita se sele
3. **EL**: selecciona el campo para el cual quiere registrar el laboreo



- Los puntos de entrada de las APIs son aquellos que exponen los endpoints. En este caso, el controller para Java o la View para Python.
- En ese sentido, nuestro punto de entrada, o *boundary*, queda representado por esa view o controller.
- En la misma línea, quien maneja la lógica del caso de uso, y se comunica con las entidades, es el servicio, quien será nuestra clase de *control*.
- Los métodos que van del Gestor a la Pantalla se eliminan, dado que la información vuelve mediante el retorno de los métodos. Esto es una consecuencia de implementar una API REST.

Mantenemos consistencia con el diagrama

```java
@RestController
@RequestMapping("/laboreos")
public class PantAdmLaboreos {

    private final GestorLaboreos gestorLaboreos;

    public PantAdmLaboreos(GestorLaboreos gestorLaboreos) {
        this.gestorLaboreos = gestorLaboreos;
    }

    @PostMapping("/iniciar")
    public ResponseEntity<List<CampoResponse>> opcionRegLaboreoLote() {
        List<CampoResponse> campos = habilitarVentana();
        return ResponseEntity.ok(campos);
    }

    private List<CampoResponse> habilitarVentana() {
        return gestorLaboreos.nuevoLaboreo();
    }

    @PostMapping("/seleccionar-campo")
    public ResponseEntity<List<LoteResponse>> tomarSeleccionCampo(@RequestBody SeleccionarCampoRequest request) {
        List<LoteResponse> lotes = gestorLaboreos.tomarSeleccionCampo(request.getNombreCampo());
        if (lotes.isEmpty()) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }
        return ResponseEntity.ok(lotes);
    }
```

Mantenemos consistencia con el diagrama

Cómo aplicamos el patrón experto?

```java
public class GestorLaboreos {

    public List<CampoResponse> nuevoLaboreo() {
        return buscarCampos();
    }

    public List<CampoResponse> buscarCampos() {
        return campos.stream()
                .filter(Campo::estaHabilitado)
                .map(campo -> new CampoResponse(campo.getNombre(), campo.getCantidadHectareas()))
                .collect(Collectors.toList());
    }

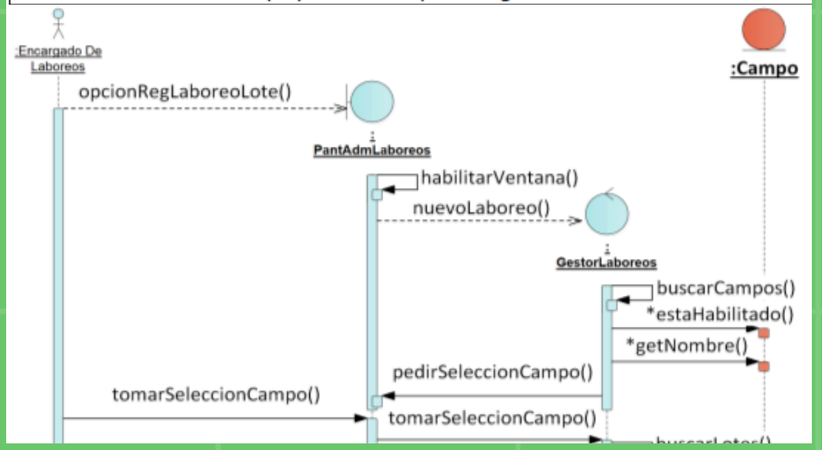
    public List<LoteResponse> tomarSeleccionCampo(String nombreCampo) {
        Campo campo = campos.stream()
                .filter(c -> c.getNombre().equals(nombreCampo))
                .findFirst()
                .orElse(null);

        if (campo == null) {
            return new ArrayList<>();
        }

        this.campoSeleccionado = campo;
        return buscarLotes(campo);
    }
```

Implementación del Caso de Uso

Flujo Básico
1. **EL**: selecciona la opción **Registrar Laboreos en un Lote**
2. **Sistema**: muestra el nombre de todos los campos habilitados y solicita se sele
3. **EL**: selecciona el campo para el cual quiere registrar el laboreo



- Conceptualmente, no es lo mismo hacer campo.getHabilitado() y validar en el gestor que hacer campo.estaHabilitado().
- Buscamos que los objetos colaboren entre sí. El Gestor no busca la información y resuelve él si está o no habilitado el campo. El mismo Campo tiene la información para determinar eso que necesitamos.
- Este mismo concepto aplica a todos los escenarios que veremos a continuación.

```java
public class Campo {
    private String nombre;
    private Double cantidadHectareas;
    private Boolean habilitado;
    private List<Lote> lotes;

    public Campo(String nombre, Double cantidadHectareas, Boolean habilitado, List<Lote> lotes) {…

    public String getNombre() {…

    public Double getCantidadHectareas() {…

    public Boolean estaHabilitado() {
        return habilitado;
    }
```

```java
public class GestorLaboreos {

    public List<CampoResponse> nuevoLaboreo() {
        return buscarCampos();
    }

    public List<CampoResponse> buscarCampos() {
        return campos.stream()
                .filter(Campo::estaHabilitado)
                .map(campo -> new CampoResponse(campo.getNombre(), campo.getCantidadHectareas()))
                .collect(Collectors.toList());
    }

    public List<LoteResponse> tomarSeleccionCampo(String nombreCampo) {
        Campo campo = campos.stream()
                .filter(c -> c.getNombre().equals(nombreCampo))
                .findFirst()
                .orElse(null);

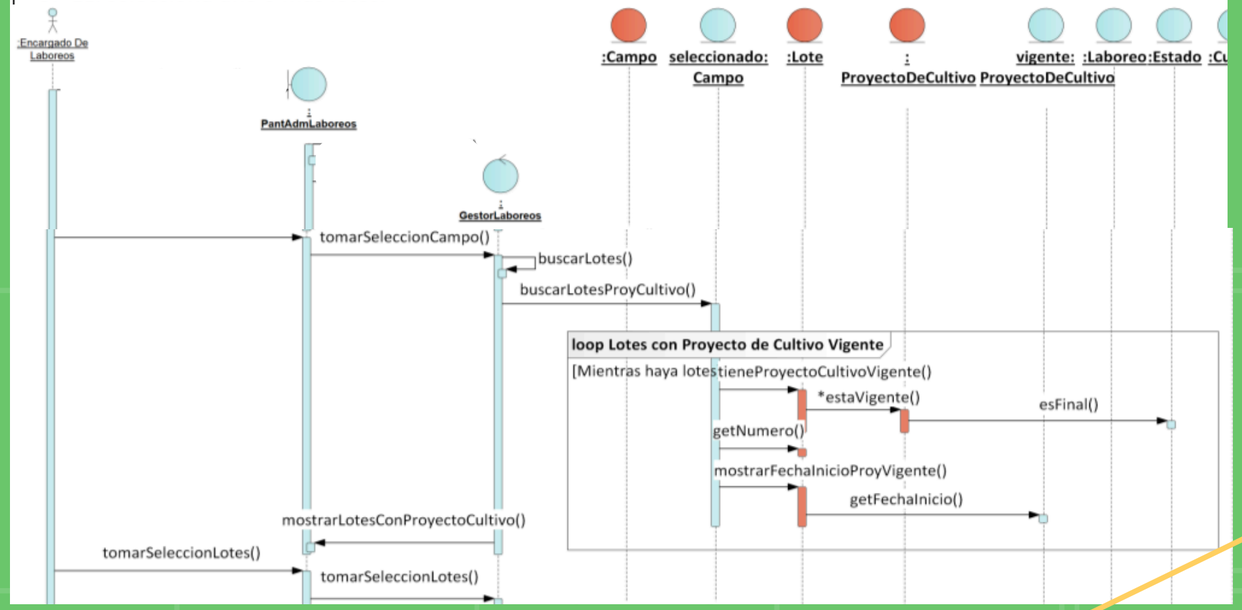
        if (campo == null) {
            return new ArrayList<>();
        }

        this.campoSeleccionado = campo;
        return buscarLotes(campo);
    }
```

- Notar que no devolvemos la entidad al boundary, devolvemos una representación en datos primitivos de la información que necesita.
- Esto puede ser un DTO (como en este caso) o un array de valores, o un valor puntual.

Implementación del Caso de Uso

4. **Sistema**: busca los lotes que corresponden a ese campo que tengan Proyecto de Cultivo vigente (significa que no esté en estado final) y encuentra al menos uno. De cada uno de ellos muestra el número de lote y la fecha de Inicio del proyecto vigente y solicita que se seleccionen los lotes para los que quiere registrar laboreos.
5. **EL**: selecciona uno o más lotes.




```java
public class GestorLaboreos {

    public List<LoteResponse> tomarSeleccionCampo(String nombreCampo) {
        Campo campo = campos.stream()
                .filter(c -> c.getNombre().equals(nombreCampo))
                .findFirst()
                .orElse(null);

        if (campo == null) {
            return new ArrayList<>();
        }

        this.campoSeleccionado = campo;
        return buscarLotes(campo);
    }

    private List<LoteResponse> buscarLotes(Campo campo) {
        return campo.buscarLotesProyCultivo().stream()
                .map(Lote -> {
                    LocalDate fechaInicio = Lote.mostrarFechaInicioProyVigente();
                    return new LoteResponse(Lote.getNumero(), fechaInicio);
                })
                .collect(Collectors.toList());
    }
```

¿Quién debe buscar los lotes?

```java
public class Campo {
    public List<Lote> buscarLotesProyCultivo() {
        return lotes.stream()
                .filter(Lote::tieneProyectoCultivoVigente)
                .collect(Collectors.toList());
    }
```

```java
public class Lote {
    public Boolean tieneProyectoCultivoVigente() {
        return proyectosCultivo.stream()
                .anyMatch(ProyectoDeCultivo::estaVigente);
    }
```

```java
public class ProyectoDeCultivo {
    public Boolean estaVigente() {
        return estado != null && !estado.esFinal();
    }
```

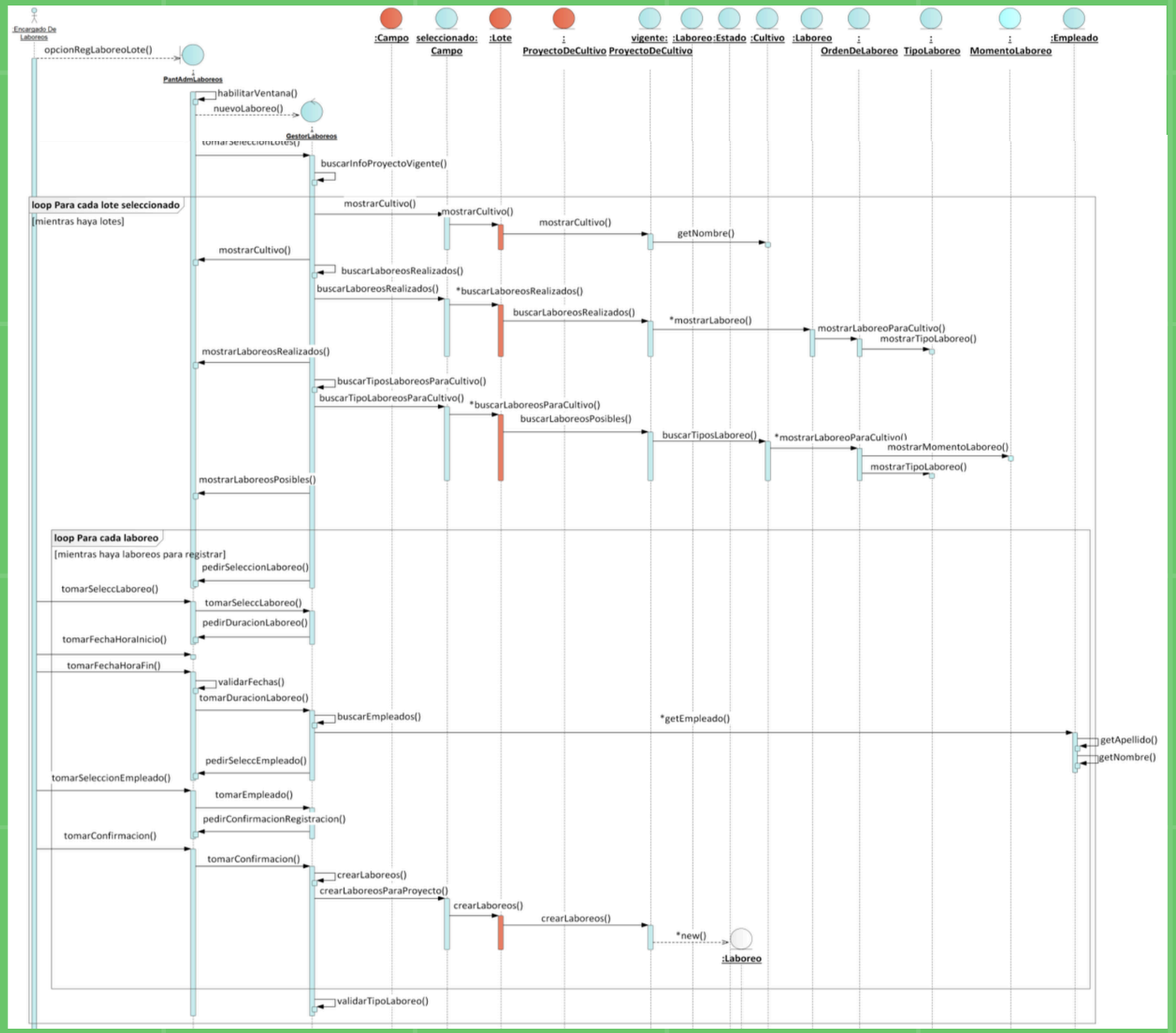
```java
public class Estado {
    public Boolean esFinal() {
        return estadoFinal;
    }
}
```

Implementación del Caso de Uso

- El resto de la implementación sigue la misma mecánica: los objetos deben colaborar entre sí para resolver lo que se requiere funcionalmente.
- A continuación, analizaremos algunas decisiones de implementación por las que se ha optado para simplificar esta demostración

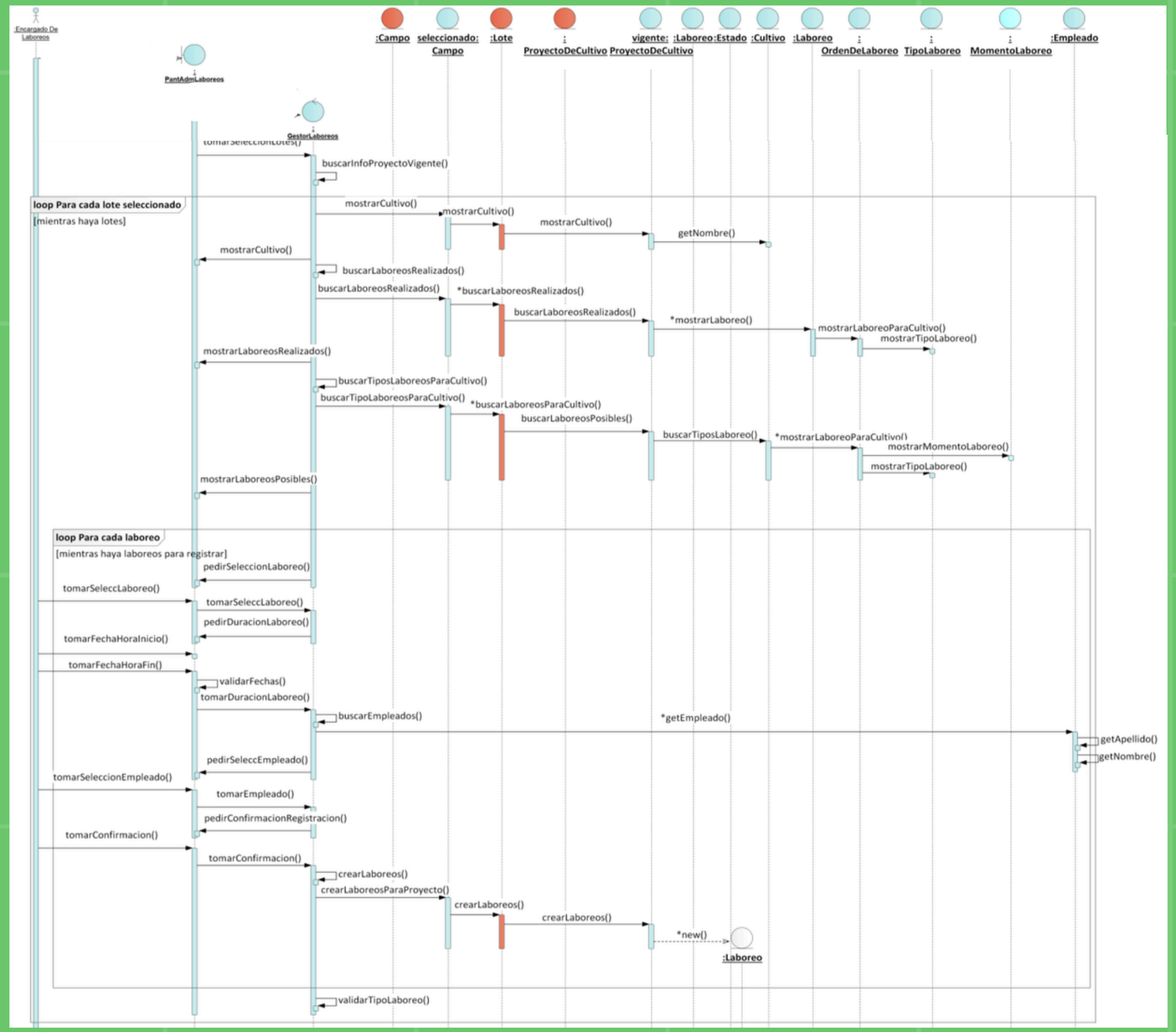


Implementación del Caso de Uso



- La realización plantea tener un fragmento de loop que envuelva los siguientes paso para cada lote seleccionado.
- Una **alternativa** es tomar la información en batch y manejar los fragmentos de loop de manera interna, en los métodos que buscan el cultivo/los laboreos realizados/ los laboreos disponibles.
- Dado que estamos implementando una API, hacer llamadas a la misma API dentro del ciclo resulta poco eficiente.
- En consecuencia, modificaremos el esquema original para realizar una única llamada en todas las ocasiones que nos envíe en lote toda la información que necesitamos para crear los laboreos.
- Naturalmente, si vamos a tomar esta decisión, debemos cambiar el diagrama, dado que debemos mantener la **consistencia** en todos los escenarios.

Implementación del Caso de Uso



Tendremos entonces:

- Método **tomarSeleccionLotes()** que devuelve para los lotes seleccionados:
  - El cultivo
  - Los laboreos realizados
  - Los tipos de laboreos disponibles para ese cultivo
- Método **tomarSeleccionLaboreo()** que nos indica qué laboreos se realizarán
- Método **tomarDuracionLaboreo()** que nos indica la duración de cada laboreo
- Método **tomarEmpleado()** que nos indica qué empleado realiza qué laboreo
- Método **tomarConfirmacion()** que confirma el registro de los laboreos

Implementación del Caso de Uso

```java
public class PantAdmLaboreos {
    @PostMapping("/seleccionar-lotes")
    public ResponseEntity<List<LoteInfoResponse>>
    tomarSeleccionLotes(@RequestBody SeleccionarLotesRequest
    request) {
        List<LoteInfoResponse> lotesInfo = gestorLaboreos.
        tomarSeleccionLotes(request.getNumerosLote());
        if (lotesInfo.isEmpty()) {
            return ResponseEntity.status(HttpStatus.
            NOT_FOUND).build();
        }
        return ResponseEntity.ok(lotesInfo);
    }
```

Mantenemos consistencia con el diagrama

```java
public class GestorLaboreos {
    public List<LoteInfoResponse> tomarSeleccionLotes(List<Integer> numerosLote) {
        if (campoSeleccionado == null) {
            return new ArrayList<>();
        }
        // Buscar en los lotes del gestor (asumimos que ya son del campo seleccionado)
        this.lotesSeleccionados = lotes.stream()
                .filter(lote -> numerosLote.contains(lote.getNumero()))
                .collect(Collectors.toList());

        return buscarInfoProyectoVigente(lotesSeleccionados);
    }

    public List<LoteInfoResponse> buscarInfoProyectoVigente(List<Lote> lotes) {
        return lotes.stream()
                .map(lote -> {
                    String cultivoNombre = campoSeleccionado.mostrarCultivo(lote);
                    if (cultivoNombre == null) {
                        return null;
                    }
                    List<LaboreoResponse> laboreosRealizados = buscarLaboreosRealizados(lote);
                    List<TipoLaboreoResponse> tiposLaboreoDisponibles = buscarTiposLaboreoParaCultivo(lote);

                    return new LoteInfoResponse(cultivoNombre, laboreosRealizados, tiposLaboreoDisponibles);
                })
                .filter(info -> info != null)
                .collect(Collectors.toList());
    }
```

Campo busca el cultivo del lote seleccionado

```java
public class Campo {
    public String mostrarCultivo(Lote lote) {
        return lote != null ? lote.mostrarCultivo
        () : null;
    }
```

```java
public class Lote {
    public String mostrarCultivo() {
        ProyectoDeCultivo proyectoVigente =
        conocerProyectoDeCultivoVigente();
        return proyectoVigente != null ? proyectoVigente.mostrarCultivo
        () : null;
    }
```

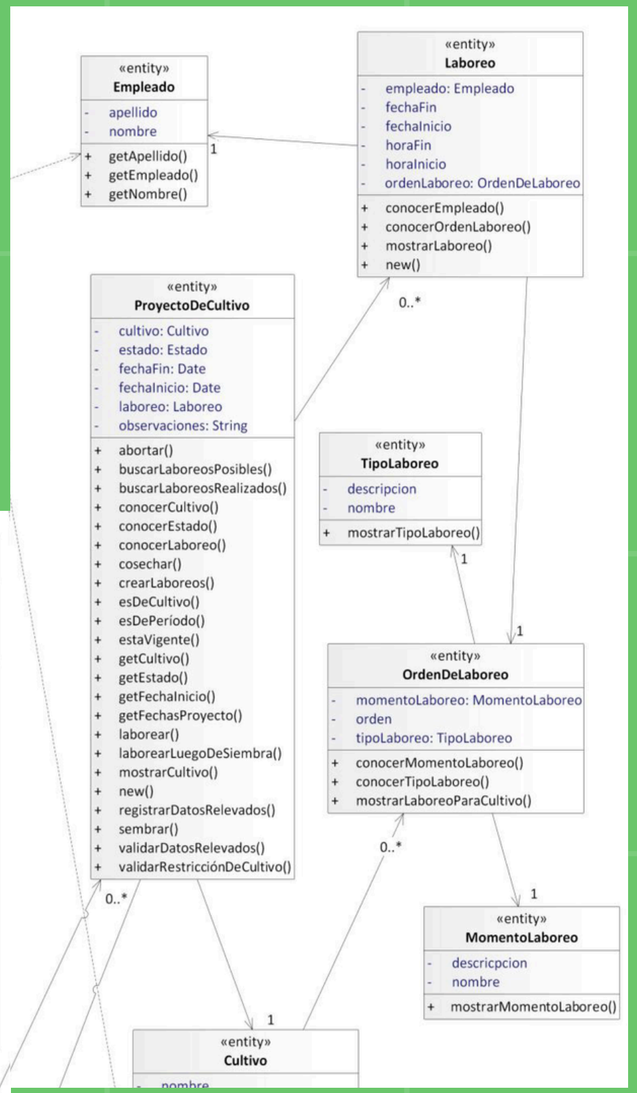
Restricción de implementación

- En el diagrama, nosotros podemos acceder al objeto vigente:ProyectoDeCultivo dado que ya hemos validado que, efectivamente, tiene uno vigente.
- Programáticamente hablando, Lote no tiene un puntero a su ProyectoDeCultivo vigente, con lo cual debemos recuperarlo cada vez que necesitemos pedirle algo.

Implementación del Caso de Uso

- Para crear un nuevo Laboreo necesitamos:
  - Las fechaHora de inicio y de fin
  - El Empleado que lo realizó
  - El OrdenDeLaboreo al que corresponde
- Necesitamos saber a qué ProyectoDeCultivo asociarlo y, por lo tanto, a qué Lote asociarlo

8. **Sistema**: **Para cada laboreo** que se desea registrar solicita que seleccione alguno.
9. **EL:** selecciona un laboreo.
10. **Sistema:** solicita se ingrese la fecha y hora de comienzo, fecha y hora de fin del laboreo.
11. **EL:** ingresa la fecha y hora de comienzo, fecha y hora de fin del laboreo.
12. **Sistema:** valida las fechas y horas de inicio y fin del laboreo y son correctos. **(ver observación 2)**
13. **Sistema**: muestra los empleados y solicita se seleccione el empleado que efectuó el laboreo.
14. **EL**: selecciona el empleado.
15. **Sistema**: **Para cada laboreo a registrar** solicita confirmación para efectuar el registro.
16. **EL:** confirma el registro.
17. **Sistema**: registra el laboreo realizado con los siguientes datos: fecha comienzo, hora comienzo, fecha fin, hora fin, empleado que lo efectuó, laboreo realizado.




```java
public class PantAdmLaboreos {

    @PostMapping("/seleccionar-lotes")
    public ResponseEntity<List<LoteInfoResponse>> tomarSeleccionLotes
    (@RequestBody SeleccionarLotesRequest request) {
        List<LoteInfoResponse> lotesInfo = gestorLaboreos.
        tomarSeleccionLotes(request.getNumerosLote());
        if (lotesInfo.isEmpty()) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }
        return ResponseEntity.ok(lotesInfo);
    }
```

```java
public class LaboreoPorLoteRequest {
    private Integer numeroLote;
    private String[] laboreo;
```

Implementación del Caso de Uso



- El caso de uso plantea que vamos a crear varios laboreos para varios lotes con proyecto de cultivo vigentes
- Necesitamos crear una estructura de datos que nos permita manejar esa información
- Vamos a construir entonces una estructura de tipo clave-valor (en este caso un Map, pero podría ser un Diccionario) que contenga:
  - Como clave, el Lote para el cual se realizará el laboreo
  - Como valor, el OrdenDeLaboreo a realizar (recodar que en el paso anterior mostramos el TipoLaboreo y el MomentoLaboreo).
- Aplicaremos la misma mecánica para la selección de la fecha y hora, y del empleado

```java
public class GestorLaboreos {
    public void tomarSeleccLaboreo(List<LaboreoPorLoteRequest> laboreosPorLote) {
        ordenesLaboreoPorLote.clear();

        if (campoSeleccionado == null || lotesSeleccionados == null || lotesSeleccionados.isEmpty()) {
            return;
        }

        Map<Integer, Lote> lotesPorNumero = lotesSeleccionados.stream()
                .collect(Collectors.toMap(Lote::getNumero, lote -> lote));

        laboreosPorLote.forEach(laboreoPorLote -> {
            Integer numeroLote = laboreoPorLote.getNumeroLote();
            String[] laboreo = laboreoPorLote.getLaboreo();

            Lote lote = lotesPorNumero.get(numeroLote);

            if (lote == null || cultivoDeLaboreo == null) {
                return;
            }

            OrdenDeLaboreo orden = cultivoDeLaboreo.conocerOrdenLaboreo().stream()
                    .filter(o -> o.conocerTipoLaboreo().getNombre().equals(laboreo[0]) &&
                            o.conocerMomentoLaboreo().getNombre().equals(laboreo[1]))
                    .findFirst()
                    .orElse(null);

            if (orden != null) {
                ordenesLaboreoPorLote.computeIfAbsent(lote, k -> new ArrayList<>()).add(orden);
            }
        });
    }
```

Implementación del Caso de Uso

- Como necesitamos saber para qué Lote y Laboreo se define esa fecha/hora de inicio y fin, podemos armar un Map que contenga:
  - Como clave, la combinación lote|tipoLaboreo|momentoLaboreo
  - Como valor, un array con las dos fechas
- Vamos a hacer lo mismo para el empleado, generando un Map con la misma clave y con el Empleado como valor

```java
public class FechaHoraPorLoteRequest {
    // Lista de fechas asociadas a cada combinación de lote + laboreo
    private List<FechaHoraPorLaboreo> fechasPorLaboreo;

    public static class FechaHoraPorLaboreo {
        private Integer numeroLote;
        private String[] laboreo; // [tipoLaboreo, momentoLaboreo]
        private LocalDateTime fechaHoraInicio;
        private LocalDateTime fechaHoraFin;
```

```java
public class GestorLaboreos {
    public List<EmpleadoResponse> tomarDuracionLaboreo(List<FechaHoraPorLoteRequest.
    FechaHoraPorLaboreo> fechasPorLaboreo) {
        if (fechasPorLaboreo == null || fechasPorLaboreo.isEmpty()) {
            return new ArrayList<>();
        }

        Map<String, LocalDateTime[]> fechasMap = new HashMap<>();

        for (FechaHoraPorLoteRequest.FechaHoraPorLaboreo fechaHora : fechasPorLaboreo) {
            // Crear clave compuesta: numeroLote|tipoLaboreo|momentoLaboreo
            String clave = generarClaveLaboreo(fechaHora.getNumeroLote(), fechaHora.
            getLaboreo());
            // Convertir a estructura simple: array con [fechaHoraInicio, fechaHoraFin]
            fechasMap.put(clave, new LocalDateTime[]{fechaHora.getFechaHoraInicio(),
            fechaHora.getFechaHoraFin()});
        }

        this.fechasPorLaboreo = fechasMap;

        return buscarEmpleados();
    }
```

```java
public class GestorLaboreos {
    public void tomarEmpleado(List<SeleccionarEmpleadoPorLaboreoRequest.EmpleadoPorLaboreo>
    empleadosPorLaboreo) {
        if (empleadosPorLaboreo == null || empleadosPorLaboreo.isEmpty()) {
            return;
        }

        Map<String, Empleado> empleadosMap = new HashMap<>();

        for (SeleccionarEmpleadoPorLaboreoRequest.EmpleadoPorLaboreo empleadoPorLaboreo :
        empleadosPorLaboreo) {
            // Buscar el empleado
            Empleado empleado = empleados.stream()
                    .filter(e -> e.getNombre().equals(empleadoPorLaboreo.getNombreEmpleado()) &&
                            e.getApellido().equals(empleadoPorLaboreo.getApellidoEmpleado()))
                    .findFirst()
                    .orElse(null);

            if (empleado != null) {
                // Crear clave compuesta: numeroLote|tipoLaboreo|momentoLaboreo
                String clave = generarClaveLaboreo(empleadoPorLaboreo.getNumeroLote(),
                empleadoPorLaboreo.getLaboreo());
                empleadosMap.put(clave, empleado);
            }
        }

        this.empleadosPorLaboreo = empleadosMap;
    }
```

- Hay otras alternativas de implementación, y otras alternativas para generar la clave del Map

```java
private String generarClaveLaboreo(Integer numeroLote, String[] laboreo) {
    return numeroLote + "|" + laboreo[0] + "|" + laboreo[1];
}
```

Implementación del Caso de Uso

1

- A este punto tenemos:
  - Un Map con lotes y laboreos
  - Un Map con laboreos/lote y fechas
  - Un Map con laboreos/lote y empleado
- Queremos saber, por lote, qué laboreos crear (con sus fechas, empleado y orden).
- Podemos tomar varios caminos:
  - Agrupar las órdenes por sus fechas y empleados en otro Map que contenga Lote y un array con todo lo demás
  - Utilizar alguna herramienta del lenguaje para simplificar la lógica
  - Otras alternativas que impliquen otras estructuras de datos
- Vamos a presentar una alternativa que genera un Map con el Lote y los laboreos a crear para ese lote

2

```java
public class GestorLaboreos {
    private void crearLaboreos() {
        // Mapa final: Lote -> Lista de arrays con información de laboreos a crear
        // Cada array contiene: [fechaInicio, fechaFin, empleado, orden]
        Map<Lote, List<Object[]>> laboreosPorLote = new HashMap<>();

        // Procesar cada lote con sus órdenes de laboreo seleccionadas
        ordenesLaboreoPorLote.forEach((lote, ordenesLaboreo) -> {
            Integer numeroLote = lote.getNumero();
            List<Object[]> laboreos = new ArrayList<>();

            // Para cada orden del lote, buscar sus fechas y empleado asociados
            for (OrdenDeLaboreo orden : ordenesLaboreo) {
                // Generar clave compuesta: "numeroLote|tipoLaboreo|momentoLaboreo"
                String clave = generarClaveLaboreo(numeroLote,
                    new String[]{orden.conocerTipoLaboreo().getNombre(), orden.conocerMomentoLaboreo().getNombre()});

                // Buscar las fechas y empleado asociados a esta orden
                LocalDateTime[] fechas = fechasPorLaboreo.get(clave);
                Empleado empleado = empleadosPorLaboreo.get(clave);

                // Si la orden tiene fechas y empleado válidos, crear array con la información
                // Array: [fechaInicio, fechaFin, empleado, orden]
                if (fechas != null && fechas.length == 2 && empleado != null) {
                    laboreos.add(new Object[]{fechas[0], fechas[1], empleado, orden});
                }
            }

            // Solo agregar el lote si tiene laboreos válidos para crear
            if (!laboreos.isEmpty()) {
                laboreosPorLote.put(lote, laboreos);
            }
        });

        // Le mandamos al Campo la información para que cree los laboreos
        campoSeleccionado.crearLaboreosParaProyecto(laboreosPorLote);
    }
```

```java
public class Campo {
    public void crearLaboreosParaProyecto(Map<Lote, List<Object[]>> laboreosPorLote) {
        // CICLO EXTERNO: Iterar sobre cada lote y su lista de laboreos
        laboreosPorLote.forEach((lote, laboreos) -> {
            // CICLO INTERNO: Para cada lote, procesar todos sus laboreos
            // Cada laboreo viene como un Object[] que necesita ser casteado a sus tipos correctos
            for (Object[] laboreoArray : laboreos) {
                // Extraer y castear cada elemento del array a su tipo correcto
                // El array tiene la estructura: [fechaInicio, fechaFin, empleado, orden]
                LocalDateTime fechaInicio = (LocalDateTime) laboreoArray[0];  // Posición 0: fecha de inicio
                LocalDateTime fechaFin = (LocalDateTime) laboreoArray[1];     // Posición 1: fecha de fin
                Empleado empleado = (Empleado) laboreoArray[2];               // Posición 2: empleado asignado
                OrdenDeLaboreo orden = (OrdenDeLaboreo) laboreoArray[3];      // Posición 3: orden de laboreo

                // Pasar los datos ya casteados al lote para que delegue la creación del laboreo
                lote.crearLaboreos(fechaInicio, fechaFin, empleado, orden);
            }
        });
    }
}
```

3

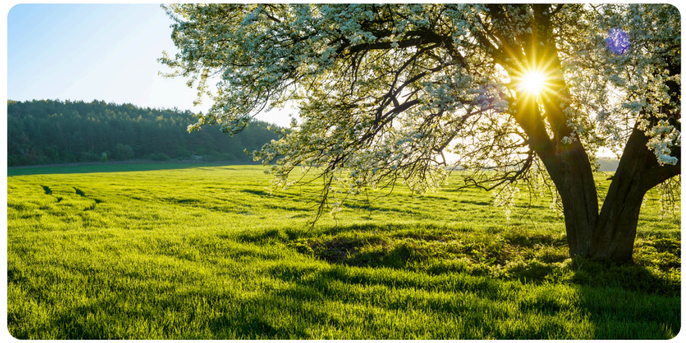
Implementación del Caso de Uso

- El resto del caso se implementa siguiendo la misma mecánica.
- La implementación es la misma en el caso de Python, adaptada a las herramientas que ofrece el lenguaje.

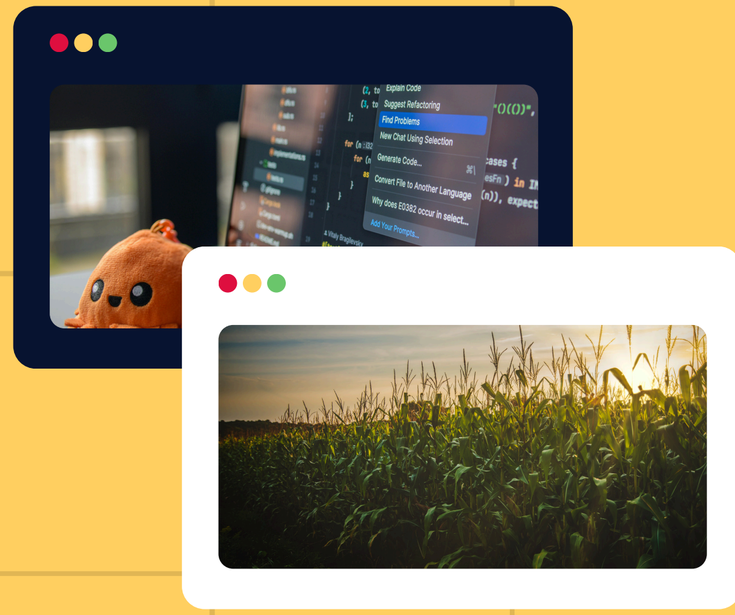


UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI

Links de Utilidad



Links de Interés



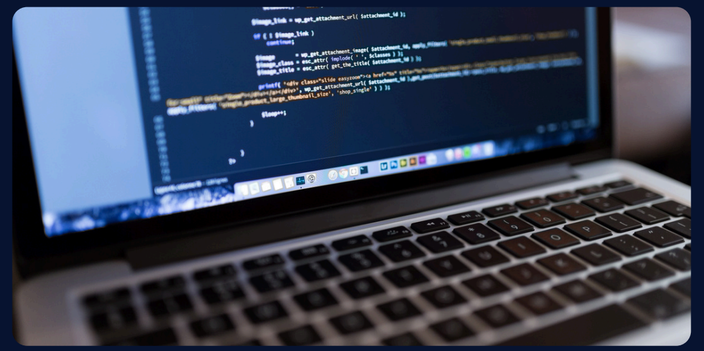
Link al Repositorio

Correo de consultas sobre esta implementación: vegamsol01@gmail.com

UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI







¡Gracias!



UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL
UTN·FRC
DSI